Simulating a Particle Detector and Particle Decay in C++

Aidan Hughes

11306492

School of Physics and Astronomy

University of Manchester

Third Year Code Report

May 2026

Abstract

The aim of this project was to utilize object oriented programming practices to create a particle detector and particle decay simulators in C++. It consists of two main inheritance chains, one for the particle classes and another for sub-detector classes. These classes, and their respective container classes ParticleBunch and Detector, practice encapsulation by storing data members as private or protected with access via public member functions. The use of virtual and pure virtual functions in the abstract parent classes of the hierarchy enables polymorphism in the child classes.

1 Introduction

The aim of this project was to create a C++ library that simulates a particle detector and its detecting functionality through the use of object oriented programming. As an addition, this code also aims to simulate particle decays within an accelerator. Particle detectors, such as ATLAS or the CMS, compose of multiple sub-detector types, each of which measure different types of particles.
The tracker, the first sub-detector a particle will pass through, uses magnetic fields to measure particle paths, so can only measure charged particles. Next are the calorimeters, which are split into two, an electromagnetic and a hadronic calorimeter. The electromagnetic calorimeter causes electrons and photons to trigger a particle shower which is measured to finds their energy. The hadronic calorimeter does the same but for protons and neutrons. Finally, due to muons high energy, they are not turned into showers by the calorimeters, and pass through to the Muon Spectrometer sub-detector. Using information about which sub-detectors were triggered, the type of particle that passed through can be deduced. These sub-detectors also have a detection efficiency associated with them, which compares the number of particles detected to the number produced during an event, usually represented as a percentage.
Along with the particles mentioned above, tau and Higgs particles are also found in accelerators, but decay quickly to other particles. This project focuses on the decays to muon and neutrinos for tau and di-photon decay for the Higgs. All particles have a four-momentum, which is a relativistically conserved property, akin to conservation of three-momentum in classical mechanics. Many particles also have masses, charges or even type specific conservation numbers like lepton number.
To simulate the sub-detectors and particles in C++, an inheritance chain was established for each.

2 Code Implementation

Object oriented programming (OOP) is built on four pillars: encapsulation, inheritance, polymorphism and abstraction.

2.1 FourMomentum

The FourMomentum class is constructed from 4 doubles corresponding to each four-momentum component: energy and the x, y and z momenta. The four-momentum components were stored as private double member variables within the FourMomentum class, as this is a more readable format then a vector in which the index corresponding to each property is not explicit to anyone extending the class. The components are accessed via getters and setters, returning the value as a double, and the setters taking a double as input and changing the member variable. Using getters and setters along with private variables is an example of encapsulation. In this case the encapsulation allowed for value checking on inputted energy, Listing 1, to ensure it was positive,

throwing an error if not, which would not have been possible if the data member was publicly accessible.

```
30 // Setter
31 void FourMomentum::set_energy(double energy)
32 {
33   if(!validate_energy(energy))
34   {
35     throw std::invalid_argument("negative energy");
36   }
37   energy_ = energy;
38 }
```

Listing 1: Showing setter for energy private data member in FourMomentum.cpp.

A major benefit of constructing separate class to hold particle four momentum is to be able to overload operators, such as the addition (+) and increment (+=) operators shown in Listing 2, to provide functions more suitable to the class. The addition operator now adds two four-momenta component-wise and returns the result as a new FourMomentum object, acting as a form of static (compile-time) polymorphism. The advantage of this can be seen in the overloaded increment operator, where the addition operator is used to add the second FourMomentum object's components into the first's, utilizing the 'this' pointer, which always points to the calling object.

```
39 // Addition
40 FourMomentum FourMomentum::operator+(const FourMomentum &vector) const
41 {
42   FourMomentum result(energy_ + vector.energy_, x_momentum_
43     + vector.x_momentum_, y_momentum_ + vector.y_momentum_,
44                 z_momentum_ + vector.z_momentum_);
45   return result;
46 }
47 // Increment
48 FourMomentum & FourMomentum::operator+=(const FourMomentum &vector)
49 {
50   *this = *this + vector;
51   return *this;
52 }
```

Listing 2: Code snippet from FourMomentum.cpp showing the overloading of + and += operators to add FourMomentum objects component-wise.

Listing 2 also showcases the use of const and references, used widely in this project. First, the FourMomentum object, vector, is passed to the function as a const reference, which means a copy of memory address of the vector is passed to the function, rather than a copy of the object. This saves on memory usage as objects are much larger than their addresses, so take up more memory when copied. However, referencing allows the underlying object to be changed by the

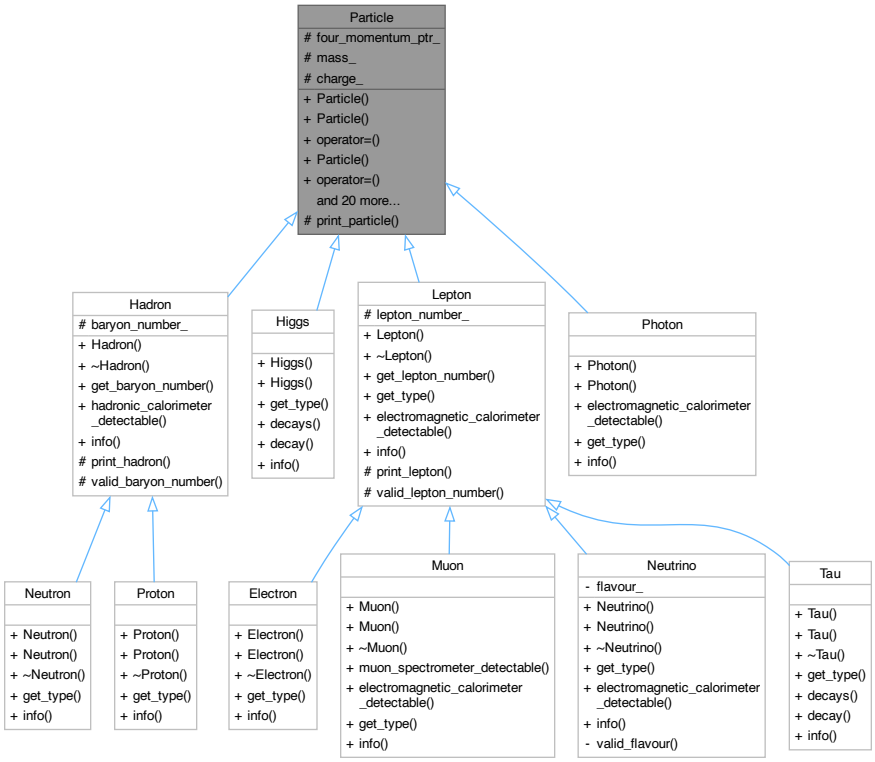

Figure 1: A UML class diagram for the Particle class inheritance hierarchy. Produced by Doxygen.

function, which is not wanted for the addition and increment operators, so const is used, which ensures the object is not changed by the function using it. Moreover, the addition operator is defined as a const member function by the const at the end of line 40, which prevents it from changing the calling FourMomentum object. This is not included in the increment operator as that intents to change the calling FourMomentum, instead the return is a reference to the calling FourMomentum.

The FourMomentum class also includes dot product and info member functions, which return the dot product as a double or print the components of the FourMomentum.

2.2 Particle hierarchy

To create classes for the particles, an inheritance hierarchy was used, shown in Figure 1. The Particle, Hadron and Lepton classes are abstract classes, so they cannot be instantiated, but allow child classes to inherit important data members and member functions from them.

2.3 Particle

The Particle class is constructed using a FourMomentum object, a double for mass and a integer for charge. The FourMomentum is stored as a unique pointer to the FourMomentum object, as this means only the address to the FourMomentum stored in the Particle. A smart pointer will protect against memory leaks by deleting the object and itself when it goes out of scope, unlike a raw pointer. The unique kind was chosen as only the particle can possess its own four momentum, so there is no need for other pointers to the FourMomentum, and therefore no need for a shared pointer.

A consequence of using smart pointers is the need to use the Rule of 5, that being the redefinition of the copy constructor and assignment, Listing 3, and the move constructor and assignment, for which the default was sufficient.

```
39 // Copy Constructor
40 Particle::Particle(const Particle &other_particle) : four_momentum_ptr_
41 (std::make_unique<FourMomentum>(*other_particle.four_momentum_ptr_)),
42 mass_(other_particle.mass_), charge_(other_particle.charge_){}
43 // Copy Assignment
44 Particle & Particle::operator=(const Particle &other_particle)
45 {
46   if(&other_particle == this){return *this;}
47
48   *four_momentum_ptr_ = *other_particle.four_momentum_ptr_;
49   mass_ = other_particle.mass_;
50   charge_ = other_particle.charge_;
51   return *this;
52 }
```

Listing 3: Code snippet from Particle.cpp showing the copy constructor and assignment redefinition in the Particle class.

PLAN OOP EXAMPLES encapsulation - all classes, particle probaly best inheritance - particle hierarchy polymorphism static - overloaded function - constuctors dynamic - info() function on particles - detector flags abstraction - particle type enum class

FEATURES USED enum class for particle type - validation in one place - easier to scale switch cases than if enables - re used for Neutrino flavour deque to do particle decay unordered map to do key value pairs rand() for efficiency / misdetection calulation

3 Results

```
1         int main()
2         {
3                 return 0;
4         }
```

4 Discussion

errors on detected value. Extra abstraction in detect function.